SDITO-LAB

Name : Sanjib Ghara **Roll No** : CSE/23/065 **Exam RollNo**: 11600223092

Assignment NO : 10

Problem Statement : Deploy a project from GitHub to EC2 by creating a new Security Group and configuring User Data.

AIM

To launch an EC2 instance, create and configure a new Security Group, deploy a project from GitHub using User Data script, and access the deployed application via public IP.

Prerequisites

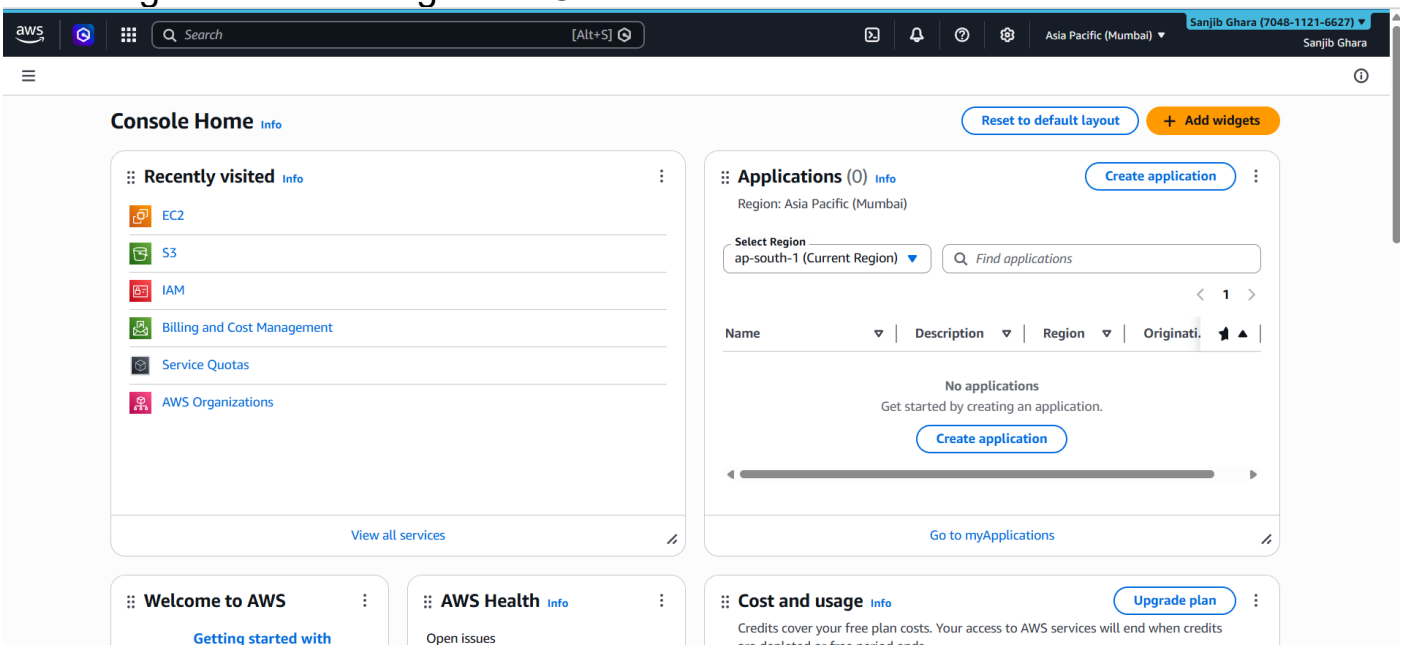
- AWS Account
- GitHub repository (public)
- Basic familiarity with Linux shell command, Git , and Node.js.
- Key Pair for EC2 login
- An AWS account with permission to create EC2 instances, security groups, and use EC2 Instance Connect.

Solution:

PHASE 1 – Create a Security Group

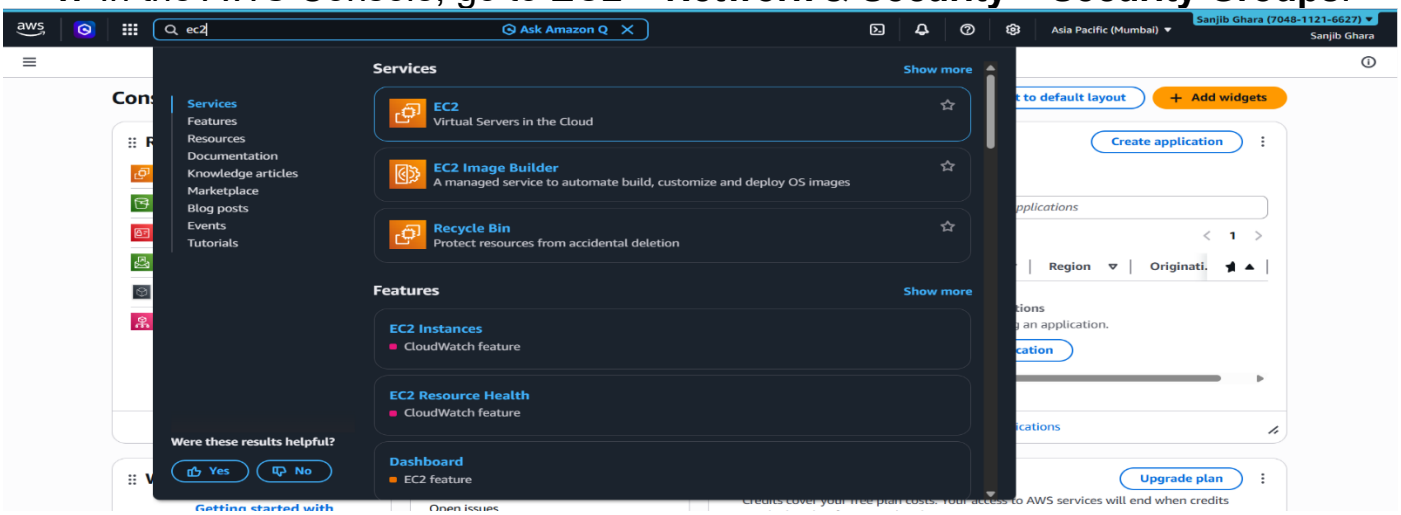
Step 1: Login to AWS Console

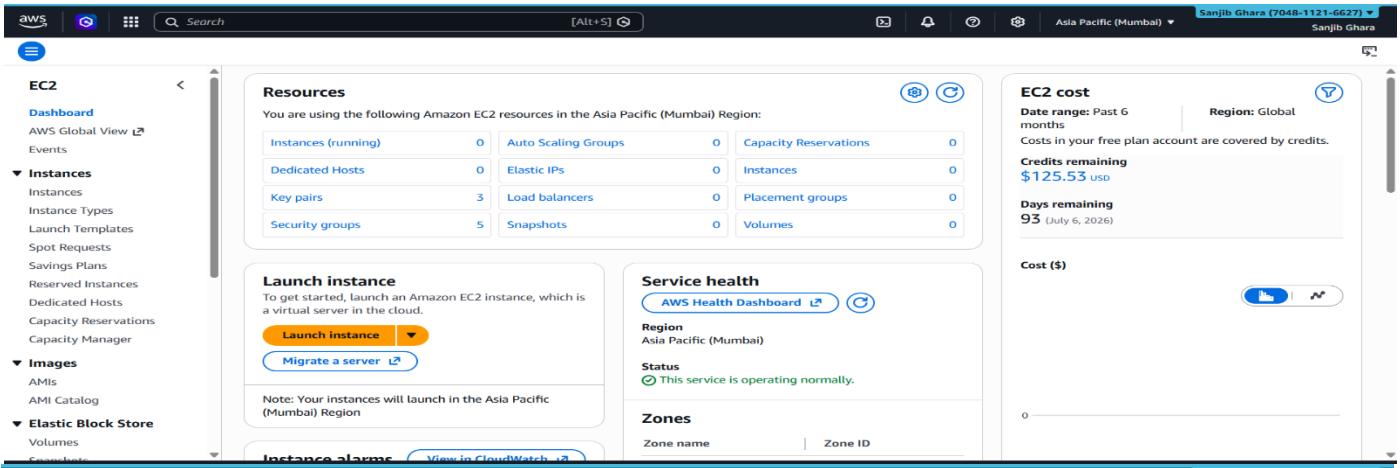
1. Login to AWS Management Console.

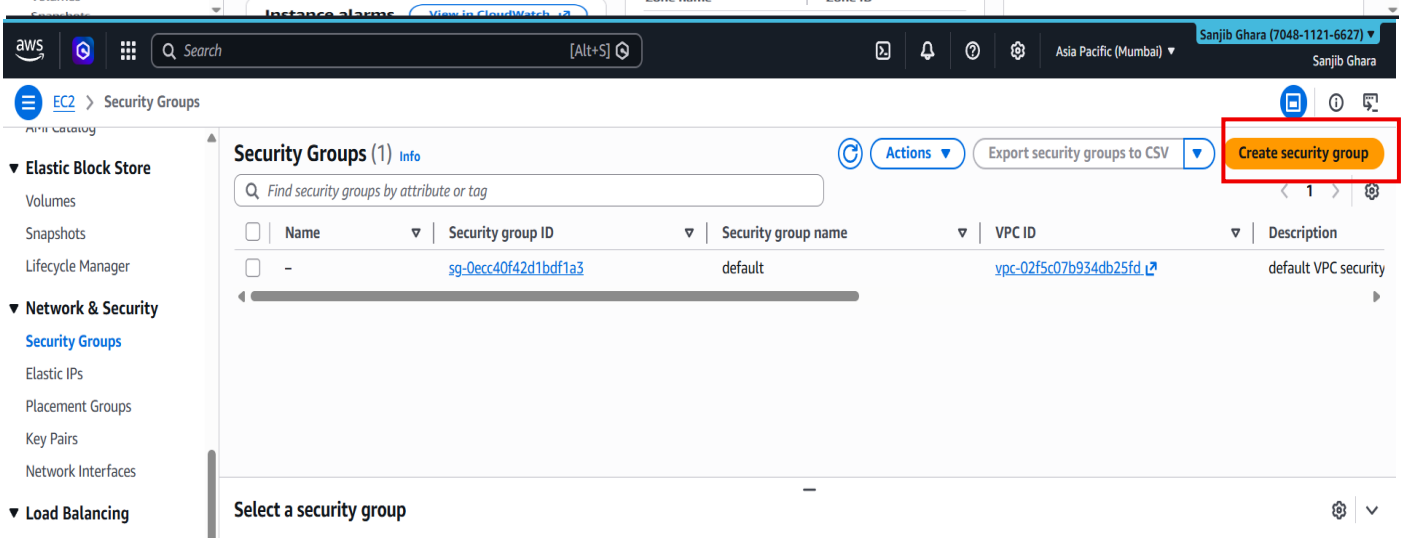


Step 2: Create a New Security Group

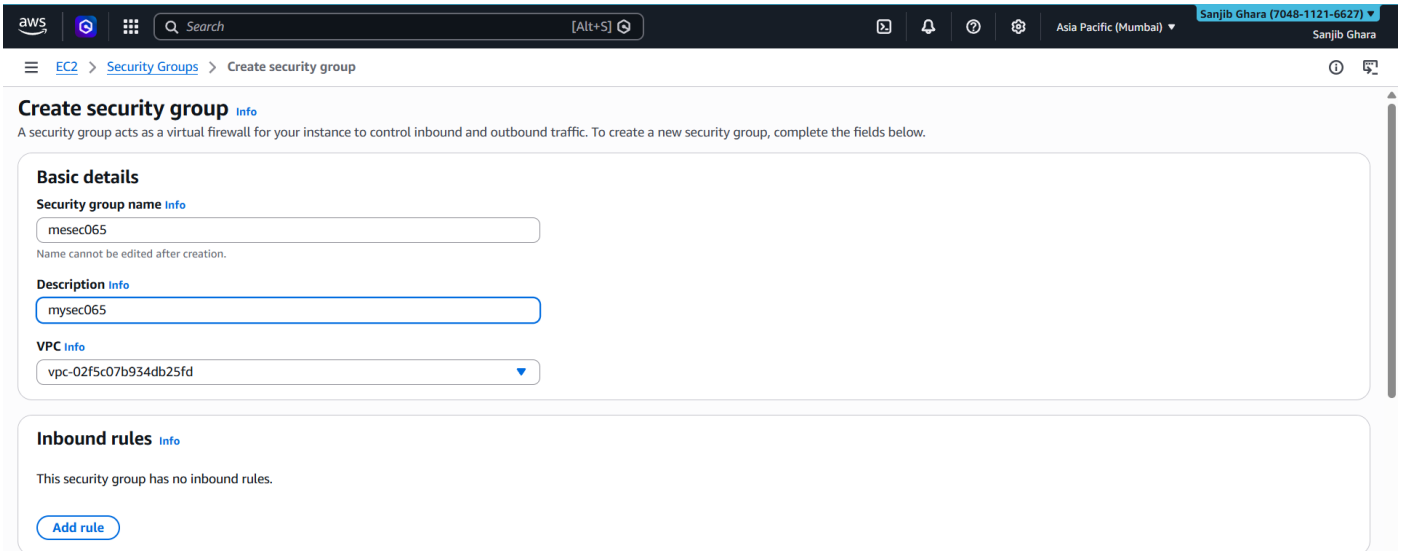
1. In the AWS Console, go to **EC2** > **Network & Security** > **Security Groups**.





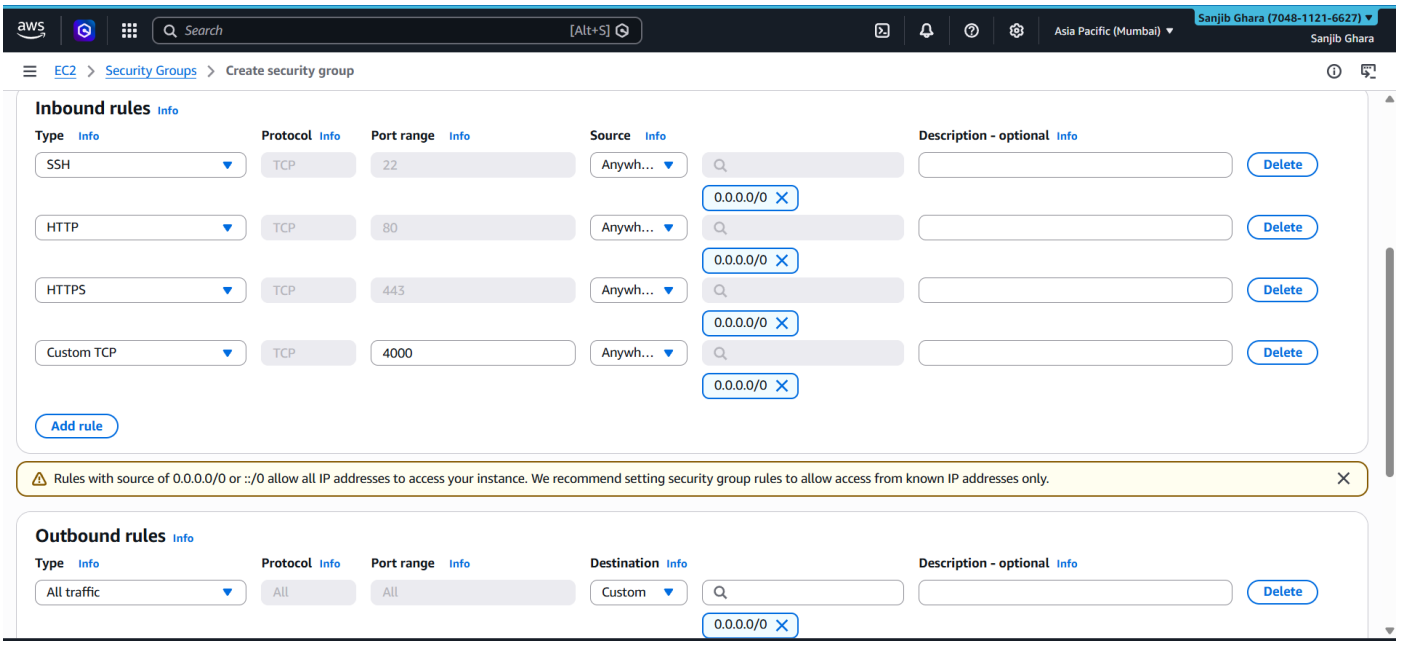


2. Click **Create security group**.
3. Fill basic details:
   - **Security group name:** mesec065
   - **Description:** mysec065
   - **VPC:** select the VPC where you will launch the instance (e.g.,vpc-02f5c07b934db25fd).

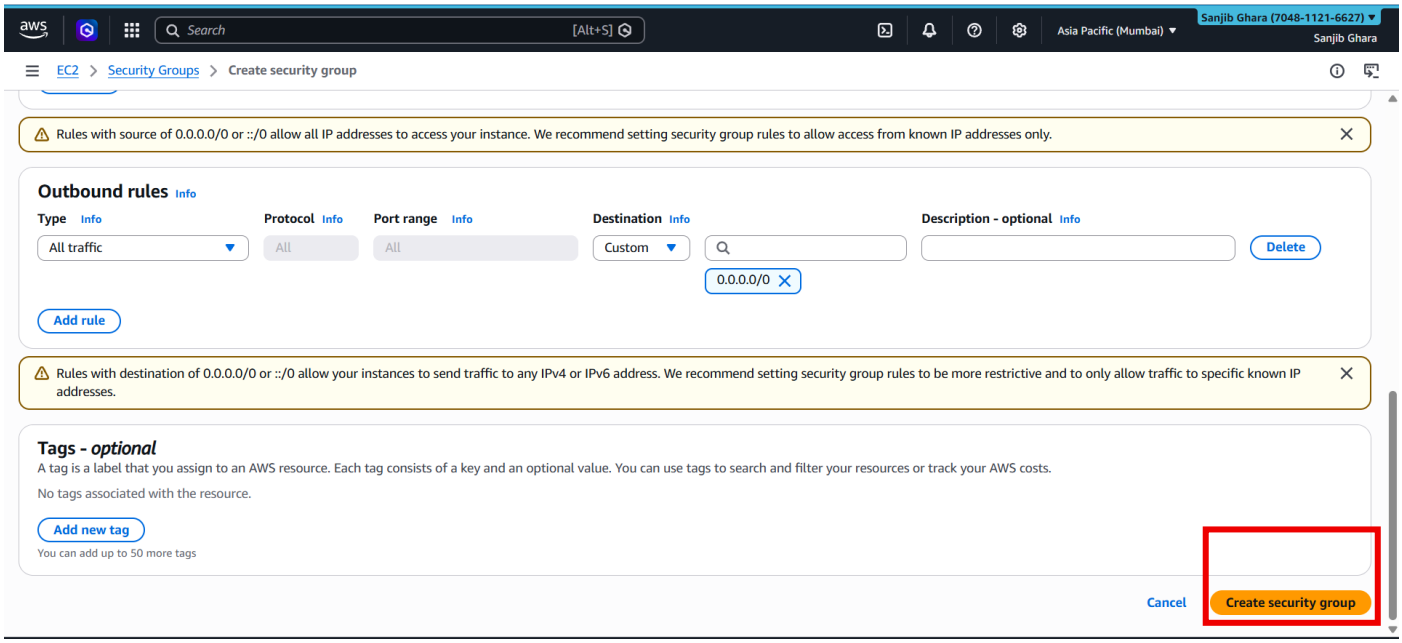


4. Click on **Add Inbound Rule** and **configure**.

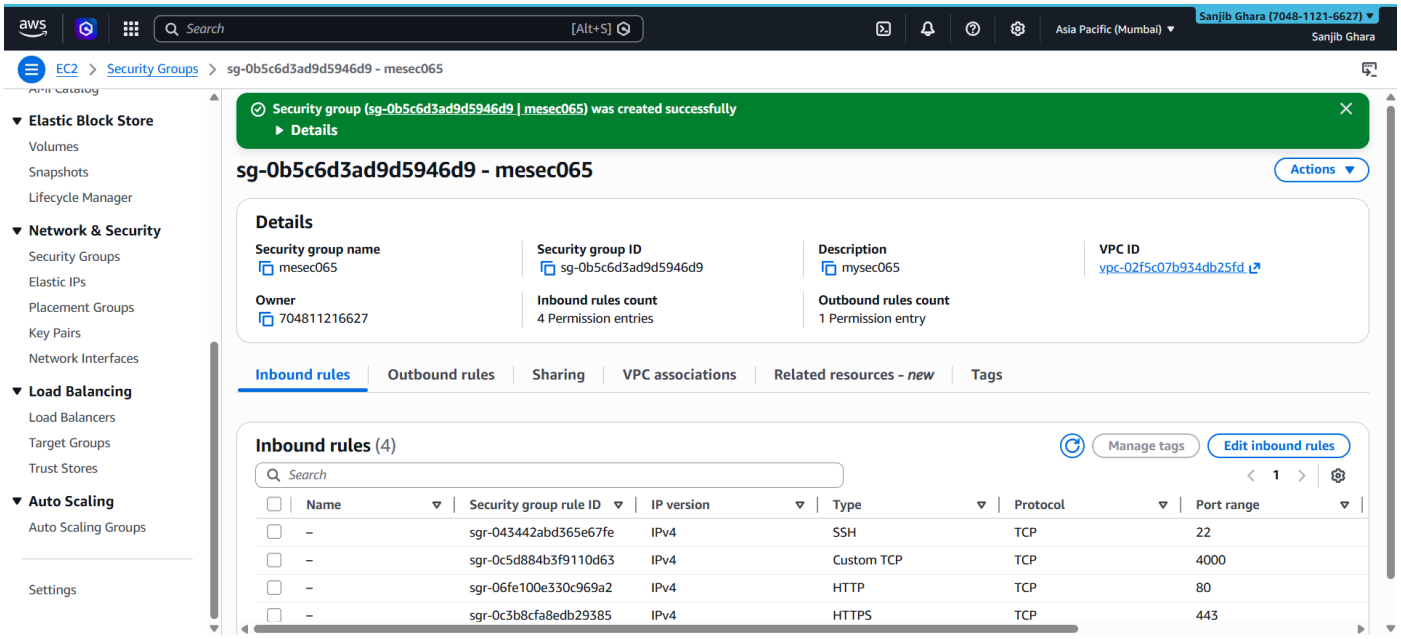
- Add **SSH** (Type: **SSH**, Protocol: **TCP**, Port: **22**) — Source: **anywhere-ipv4** (0.0.0.0/0)
- Add **HTTP** (Type: **HTTP**, **TCP**, Port: **80**) — Source: **anywhere-ipv4** (0.0.0.0/0).
- Add **HTTPS** (Type: **HTTPS**, **TCP**, Port: **443**) — Source: **anywhere-ipv4** (0.0.0.0/0).
- Add **Custom TCP** (Type: **Custom TCP**, Protocol: **TCP**, Port range: **4000**) — Source: **anywhere-ipv4** (0.0.0.0/0).



5. Configure **Outbound rules**: default is *All traffic* to 0.0.0.0/0. Leave as it is.
6. Now , Click on **Create security group**.



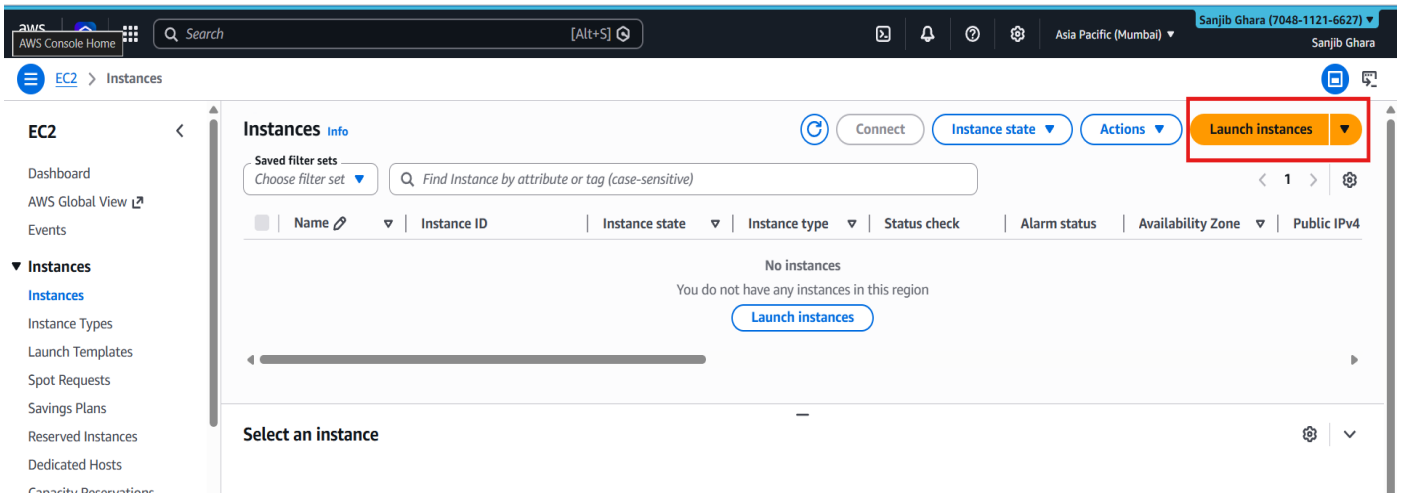
- Now we can see a success message and the new **mesec065** security group is listed.



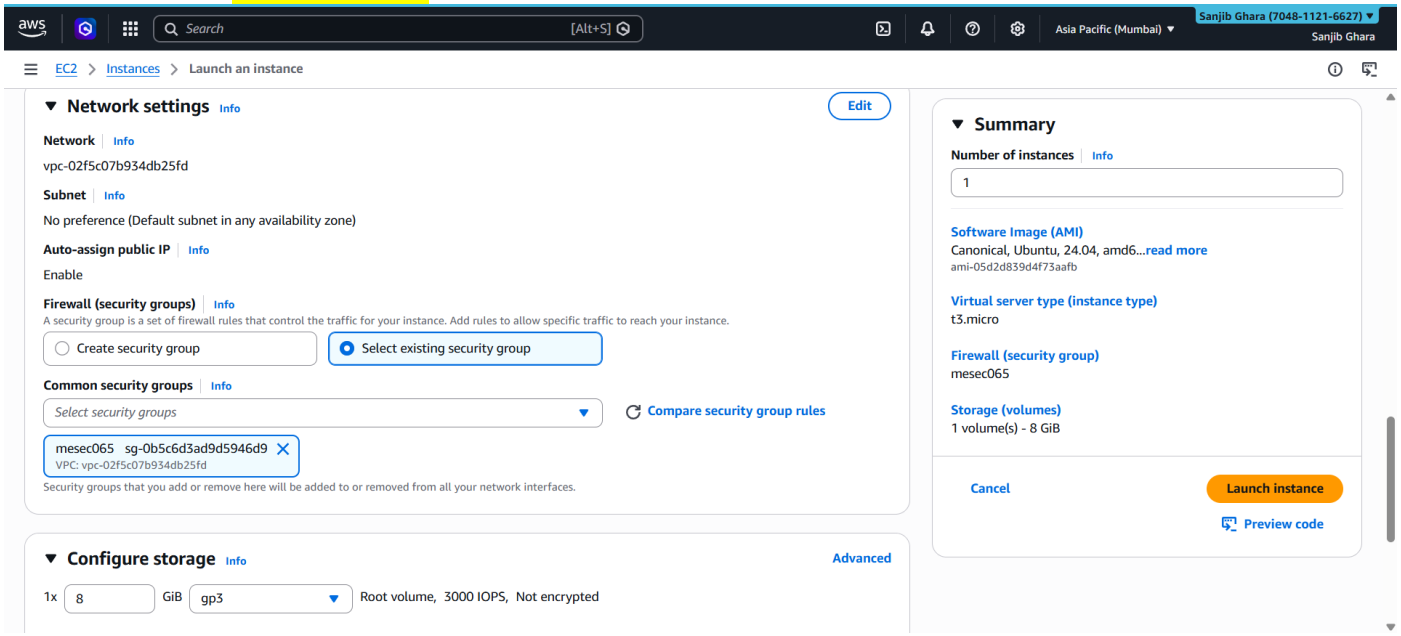
PHASE 2 – Launch an EC2 Instance and Add User Data Script

Step 1: Launch an EC2 Instance and Attach the Security Group

1. Go to **EC2** > **Instances** > **Launch instances**.



2. Choose an AMI: ***Canonical, Ubuntu 24.04*** .
3. Choose Instance Type: **t3.micro** (free-tier account) or another small type.
4. Create a key Pair in my case : **webkey1.**
5. Configure instance details:
   - **Network (VPC):** choose the **same VPC** used for the security group.
   - **Subnet:** choice **default**.
   - **Auto-assign public IP: Enable**
6. **Firewall (security groups):** choose **Select existing security group**
7. **Common Security Group :** Select the newly created Security Group. In my case its - **mesec065**.



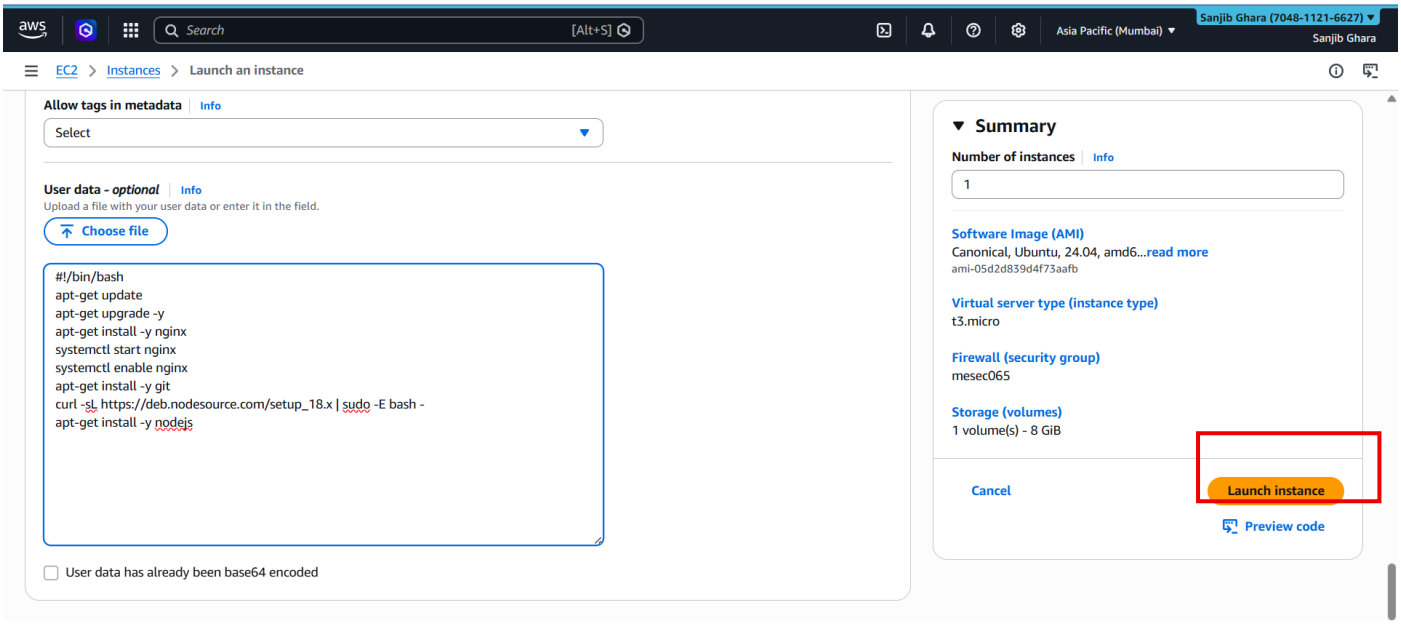
8. Scroll to **'Advanced Details'** section.

Step 2: Add User Data Script

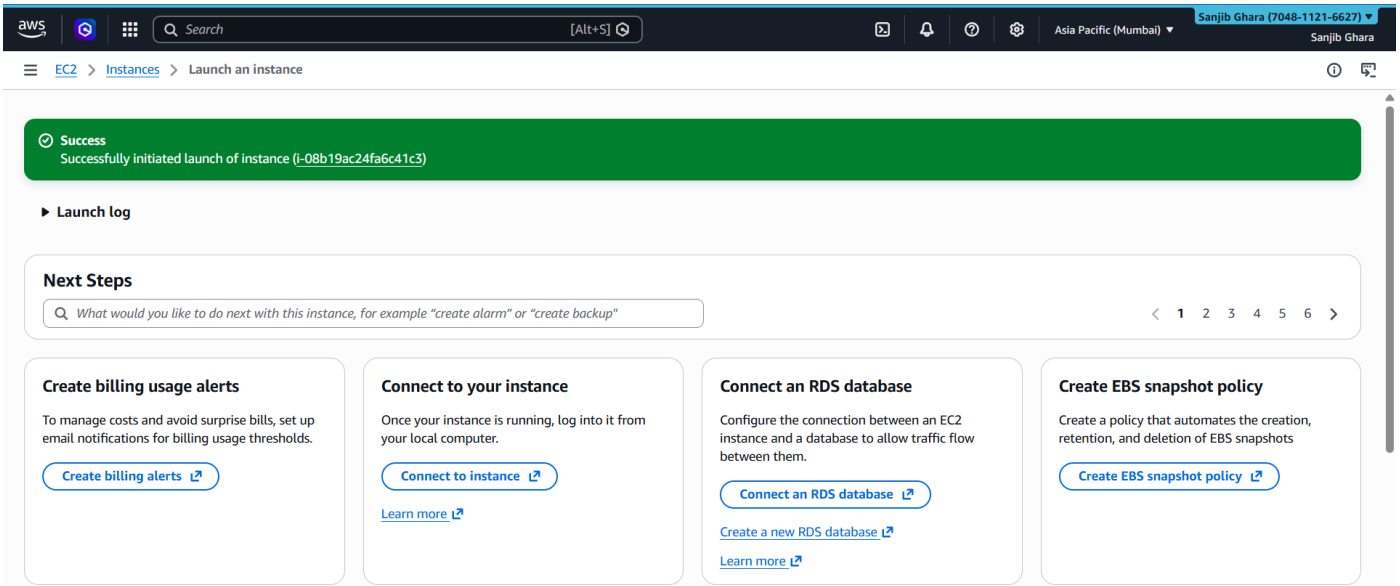
1. Click on **Advance Details** .
2. **User data:** In that, paste the following script: (below). This runs on first boot and prepares the instance.

paste in the *User data* box:

```
#!/bin/bash
apt-get update
apt-get upgrade -y
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
apt-get install -y git
curl -sL https://deb.nodesource.com/setup_18.x | sudo -E bash -
apt-get install -y nodejs
```



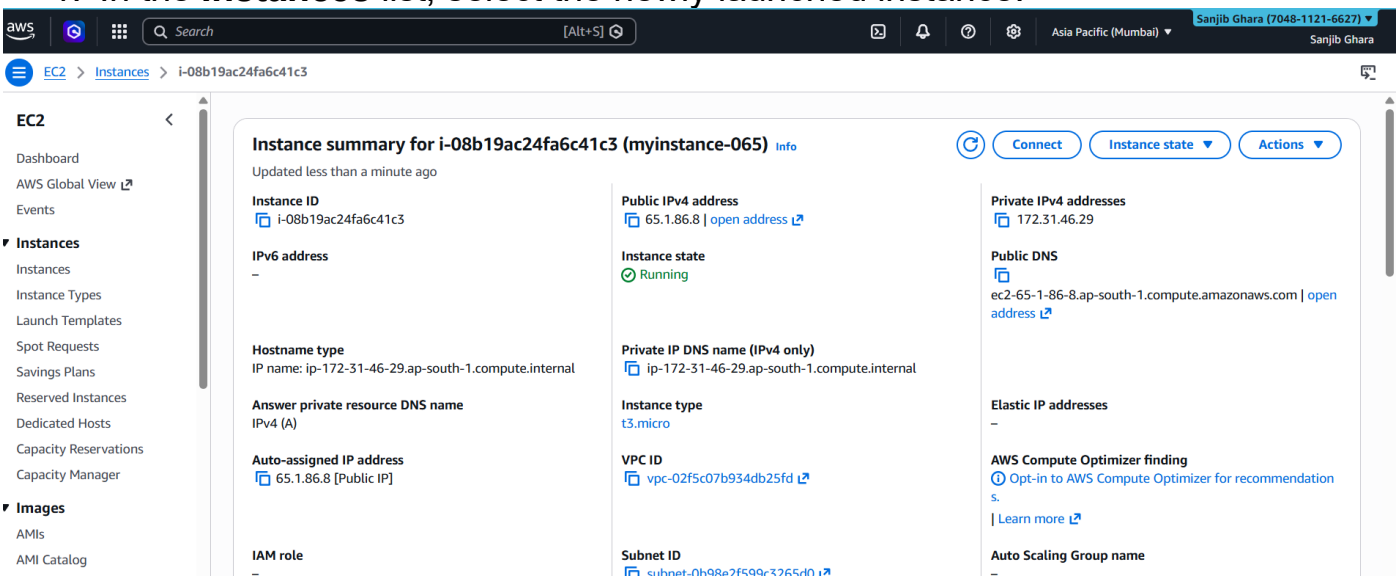
3. Click '**Launch instance**'.



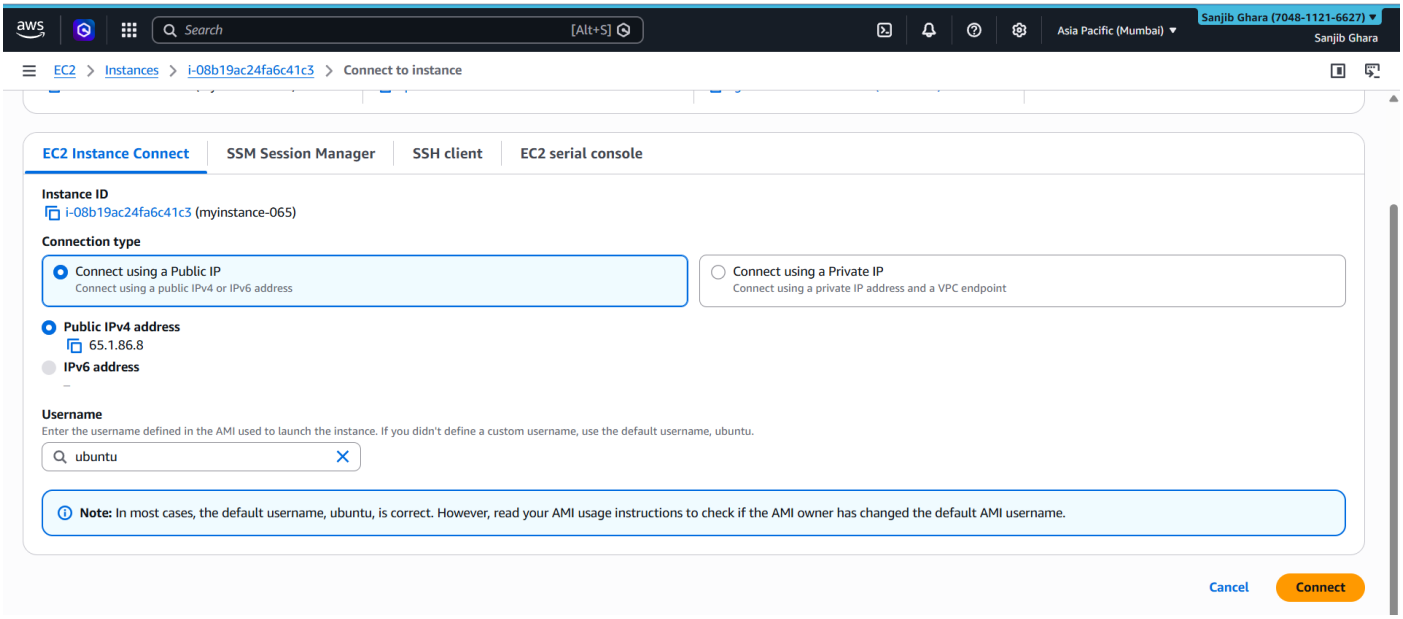
PHASE 3 – Connect to the Instance (EC2 Instance Connect)

Step 1: Connect to EC2 Instance

1. In the **Instances** list, select the newly launched instance.



2. Click **Connect** > **EC2 Instance Connect**.
3. Choose **Connect using a Public IP** (default). The **Username** for the Ubuntu AMI is **ubuntu**.
4. Click **Connect** — It will open a browser-based terminal session.

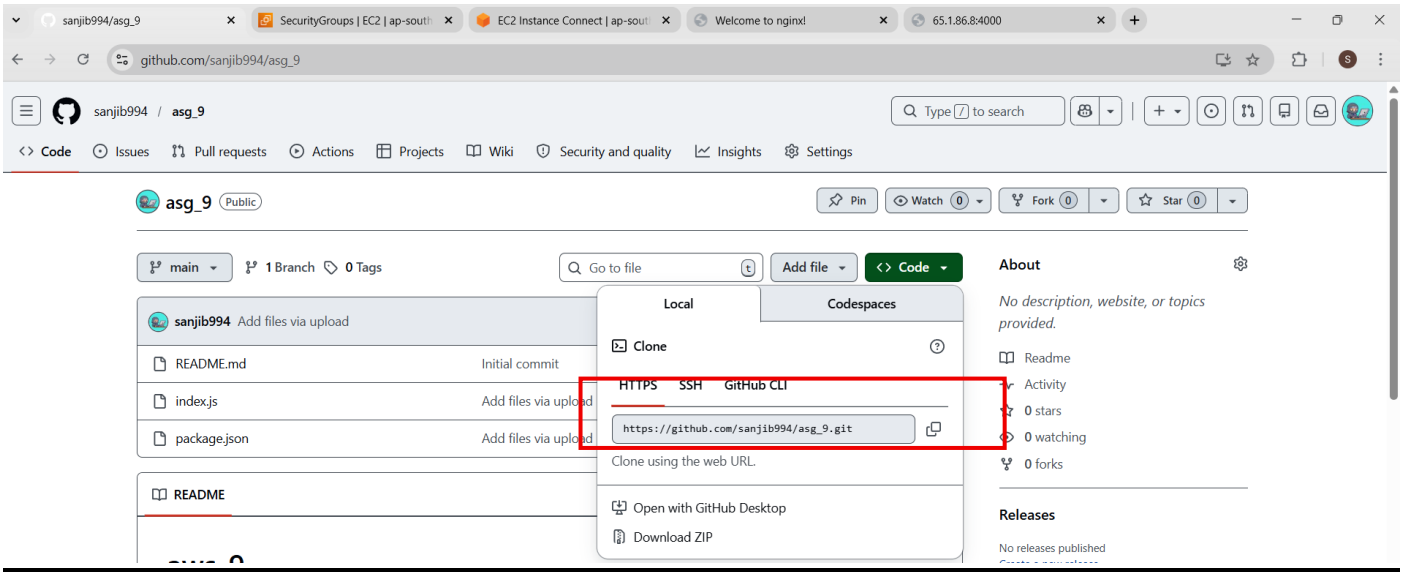


*(Then Its show successful connection and the **public IP: 65.1.86.8** and username **ubuntu**.)*

PHASE 4 – Pull the Project from GitHub and Start the Website

Step 1: Clone the REPO From GITHUB

1. Go to **GITHUB Account** .
2. Open the which have node files , in my case ‘asg_9’ , and **copy** the REPO **URL Link**.



3. Then, Once connected via Instance Connect, **perform these commands** in the **terminal:**

For clone the GitHub repo used
git clone https://github.com/sanjib994/asg_9.git

To change the directory & for checking the files that clone actually done
cd asg_9
ls

It will show
index.js package.json README.md

To Install Node dependencies
npm install

To Start the Node app
node index.js

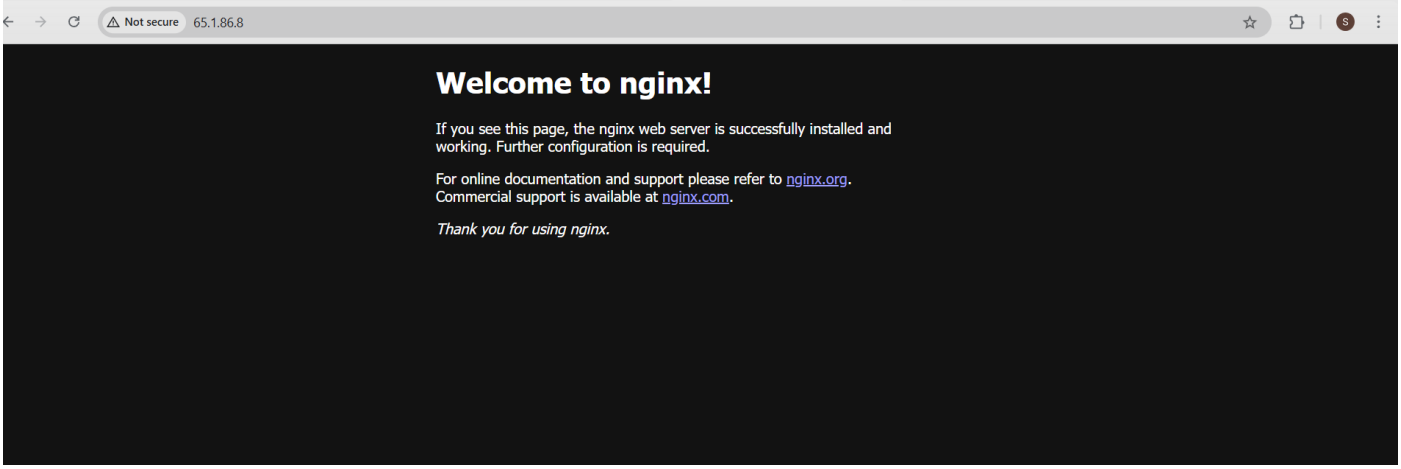
It will show Result as: **"Started server"**

```
ubuntu@ip-172-31-46-29:~$ git clone https://github.com/sanjib994/asg_9.git
Cloning into 'asg_9'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 11 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (11/11), done.
Resolving deltas: 100% (2/2), done.
ubuntu@ip-172-31-46-29:~$ cd asg_9
ubuntu@ip-172-31-46-29:~/asg_9$ ls
README.md  index.js  package.json
ubuntu@ip-172-31-46-29:~/asg_9$ npm install

added 224 packages, and audited 225 packages in 17s

26 packages are looking for funding
  run `npm fund` for details

1 low severity vulnerability

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 10.8.2 -> 11.12.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.12.1
```

```
remote: Compressing objects: 100% (9/9), done.
remote: Total 11 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (11/11), done.
Resolving deltas: 100% (2/2), done.
ubuntu@ip-172-31-46-29:~$ cd asg_9
ubuntu@ip-172-31-46-29:~/asg_9$ ls
README.md  index.js  package.json
ubuntu@ip-172-31-46-29:~/asg_9$ npm install

added 224 packages, and audited 225 packages in 17s

26 packages are looking for funding
  run `npm fund` for details

1 low severity vulnerability

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 10.8.2 -> 11.12.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.12.1
npm notice To update run: npm install -g npm@11.12.1
npm notice
ubuntu@ip-172-31-46-29:~/asg_9$ node index.js
started server
```

Step 2: Verify This From MY Browser

1. Copy the **Public IPv4 address** from **EC2 Instance dashboard**.
2. Check the default nginx page (HTTP port 80): open **http://<PUBLIC_IP>** (e.g., http:// ***65.1.86.8***) — we can see **Welcome to nginx!**



3. Check the Node.js app on port 4000: open **http://<PUBLIC_IP>:4000** (e.g., http:// ***65.1.86.8***:4000)— we can see the web output -→ **Hello MCKV**

Output :

The project hosted on GitHub is successfully deployed on AWS EC2 instance and accessible via public IP address.

Conclusion :

This lab demonstrates how to deploy a GitHub project on AWS EC2 by configuring a Security Group and using User Data automation script.